

Coverage Metric - GPU Optimized

Karina

2026-02-17

Table of contents

0.1	1. Setup e Imports	1
0.2	2. Cargar Datos	2
0.3	3. Cargar SAE y Extraer Features	3
0.4	4. Filtrar BSPs de Piezas	4
0.5	5. Implementación de Coverage	4
0.5.1	Estrategia de optimización:	4
0.6	6. Calcular Coverage	7
0.7	7. Análisis de Resultados	7
0.8	8. Guardar Resultados	10
0.9	9. Generar PDF con Quarto	10

```
# Configuración para visualización en PDF
import pandas as pd
import numpy as np
import sys

# Para que el texto de pandas no se corte
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 100) # Ancho fijo para pandas

# Para prints largos de numpy - ancho más conservador
np.set_printoptions(linewidth=100, edgeitems=3)

# Configurar ancho de terminal para prints
import os
os.environ['COLUMNS'] = '100'

print(" Configuración para PDF lista")
```

Configuración para PDF lista

0.1 1. Setup e Imports

```
import numpy as np
import torch
import sys
import os
from pathlib import Path
from tqdm import tqdm
import time

# Configurar paths
project_root = Path('../..').resolve()
sys.path.insert(0, str(project_root))
```

```

from sae.models.sae import SparseAutoencoder
from sae.tools.naming_utils import get_checkpoint_dir, get_model_name
from sae.tools.experiment_utils import get_experiment_dir, get_metrics_dir, get_report_name

# Verificar GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memoria disponible: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

```

Device: cuda
GPU: NVIDIA GeForce RTX 5060
Memoria disponible: 8.55 GB

```

# Metadata del experimento (debe coincidir con el notebook de entrenamiento)
NAMING_META = {
    'variante': 'standard',          # Arquitectura del SAE
    'tecnica': 'l1',                 # Técnica de sparsity
    'capa': 6,                       # Layer del modelo OthelloGPT
    'juegos': 1000,                  # Número de partidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt', # Path base para checkpoints
    'experiment_base_path': os.path.abspath('.././experiments')          # sae/experiments/
}

print('\n Metadata del experimento:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')

```

Metadata del experimento:
variante: standard
tecnica: l1
capa: 6
juegos: 1000
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments

0.2 2. Cargar Datos

```

# Cargar activaciones pre-SAE
activations_path = project_root / "sae" / "activations" / "data" / "layer5_200games.npy"
activations = np.load(activations_path)

print(f"Activaciones del modelo:")
print(f" Shape: {activations.shape}")
print(f" Memoria: {activations.nbytes / (1024**2):.2f} MB")

```

Activaciones del modelo:
Shape: (11800, 512)
Memoria: 23.05 MB

```

# Cargar ground truth de BSPs
bsp_gt_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.npy"
bsp_names_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.names.npy"

bsp_ground_truth = np.load(bsp_gt_path)
bsp_names = np.load(bsp_names_path, allow_pickle=True)

```

```
print(f"\nGround truth BSPs:")
print(f"  Shape: {bsp_ground_truth.shape}")
print(f"  Total BSPs: {len(bsp_names)}")
```

```
Ground truth BSPs:
  Shape: (11800, 198)
  Total BSPs: 198
```

0.3 3. Cargar SAE y Extraer Features

```
# Configuración del SAE
input_dim = 512
hidden_dim = 16384

# Construir ruta del modelo usando naming_utils
checkpoint_dir = get_checkpoint_dir(
    NAMING_META['base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa']
)
model_name = get_model_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'best'
)
model_path = checkpoint_dir / model_name

# Cargar modelo (weights_only=False: checkpoint contiene objetos Path en config)
sae = SparseAutoencoder(input_dim, hidden_dim).to(device)
checkpoint = torch.load(model_path, map_location=device, weights_only=False)
sae.load_state_dict(checkpoint['model_state_dict'])
sae.eval()

print(f"SAE cargado:")
print(f"  Path: {model_path}")
print(f"  Input: {input_dim}, Hidden: {hidden_dim}")
print(f"  Expansion: {hidden_dim/input_dim}x")
print(f"  Epoch: {checkpoint.get('epoch', 'N/A')}, Val MSE: {checkpoint.get('val_mse', float('nan')):.6f}")
```

```
SAE cargado:
  Path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_standard\sae_l1_standard\sae_standard_l1_16
  Input: 512, Hidden: 16384
  Expansion: 32.0x
  Epoch: N/A, Val MSE: nan
```

```
# Extraer features del SAE en GPU
print("Extrayendo features del SAE...")
activations_tensor = torch.from_numpy(activations).float().to(device)

with torch.no_grad():
    sae_features = torch.relu(sae.encoder(activations_tensor))

print(f"\nFeatures SAE:")
print(f"  Shape: {sae_features.shape}")
print(f"  Device: {sae_features.device}")
```

```
print(f" Sparsity: {(sae_features == 0).float().mean():.2%}")
print(f" Activaciones promedio: {(sae_features > 0).sum(dim=1).float().mean():.1f}")
```

Extrayendo features del SAE...

Features SAE:

```
Shape: torch.Size([11800, 16384])
Device: cuda:0
Sparsity: 96.54%
Activaciones promedio: 567.7
```

0.4 4. Filtrar BSPs de Piezas

Coverage solo usa las 128 BSPs de piezas (mías/oponente), sin vacías.

```
# Filtrar BSPs de piezas (terminan en '1' o '2', no en '0')
bsp_piezas_indices = []
bsp_piezas_names = []

for i, name in enumerate(bsp_names):
    if len(name) == 6 and name.startswith('BSP') and not name.endswith('0'):
        bsp_piezas_indices.append(i)
        bsp_piezas_names.append(name)

bsp_piezas_indices = np.array(bsp_piezas_indices)
bsp_piezas_gt = bsp_ground_truth[:, bsp_piezas_indices]

# Convertir a tensor en GPU
bsp_piezas_gt_tensor = torch.from_numpy(bsp_piezas_gt).bool().to(device)

print(f"BSPs de piezas para Coverage:")
print(f" Total: {len(bsp_piezas_indices)}")
print(f" Shape: {bsp_piezas_gt_tensor.shape}")
print(f" Device: {bsp_piezas_gt_tensor.device}")
print(f" Primeras 5: {'', ' '.join(bsp_piezas_names[:5])}")
print(f" Últimas 5: {'', ' '.join(bsp_piezas_names[-5:])}")
```

BSPs de piezas para Coverage:

```
Total: 128
Shape: torch.Size([11800, 128])
Device: cuda:0
Primeras 5: BSPA11, BSPA12, BSPA21, BSPA22, BSPA31
Últimas 5: BSPH62, BSPH71, BSPH72, BSPH81, BSPH82
```

0.5 5. Implementación de Coverage

0.5.1 Estrategia de optimización:

1. **Vectorización:** Procesar múltiples features en paralelo usando operaciones matriciales
2. **Batch processing:** Dividir en chunks para no saturar memoria GPU
3. **Pre-cálculo:** Calcular máximos una sola vez
4. **Early stopping:** Terminar cuando F1 = 1.0

```
def fast_f1_score_gpu(y_true, y_pred, eps=1e-8):
    """
    F1 score vectorizado en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
```

```

        y_pred: Tensor (n_positions, n_candidates) booleano

Returns:
    f1_scores: Tensor (n_candidates,)
    """
    # y_true: (n_positions,) -> expand to (n_positions, 1)
    y_true_expanded = y_true.unsqueeze(1) # (n_positions, 1)

    # Calcular TP, FP, FN
    tp = (y_true_expanded & y_pred).sum(dim=0).float() # (n_candidates,)
    fp = (~y_true_expanded & y_pred).sum(dim=0).float()
    fn = (y_true_expanded & ~y_pred).sum(dim=0).float()

    # Precision y Recall
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)

    # F1
    f1 = 2 * (precision * recall) / (precision + recall + eps)

    return f1

def calculate_coverage_gpu_optimized(
    sae_features, # Tensor (n_positions, n_features) en GPU
    bsp_ground_truth, # Tensor (n_positions, n_bsps) en GPU, bool
    thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
    batch_size_features=512, # Procesar 512 features a la vez
    verbose=True
):
    """
    Calcula Coverage de forma optimizada para GPU.

    Procesa múltiples features y thresholds en paralelo usando operaciones matriciales.
    """
    n_positions, n_features = sae_features.shape
    n_bsps = bsp_ground_truth.shape[1]
    n_thresholds = len(thresholds)

    if verbose:
        print("="*60)
        print("CALCULATING COVERAGE (GPU OPTIMIZED)")
        print("="*60)
        print(f"Positions: {n_positions:,}")
        print(f"SAE Features: {n_features:,}")
        print(f"BSPs: {n_bsps:,}")
        print(f"Thresholds: {n_thresholds:,}")
        print(f"Batch size (features): {batch_size_features:,}")
        print(f"Total evaluations: {n_bsps * n_features * n_thresholds:,}")
        print("="*60)

    # Pre-calcular máximos de features
    f_max = sae_features.max(dim=0)[0] # (n_features,)
    active_features = (f_max > 0).nonzero(as_tuple=True)[0]
    n_active = len(active_features)

    if verbose:
        print(f"\nActive features: {n_active:,}/{n_features:,} ({n_active/n_features*100:.1f}%)")
        print(f"Reduced evaluations: {n_bsps * n_active * n_thresholds:,}")
        print()

    # Almacenar resultados

```

```

best_f1s = torch.zeros(n_bsps, device=sae_features.device)
best_features = torch.full((n_bsps,), -1, dtype=torch.long, device=sae_features.device)
best_thresholds = torch.full((n_bsps,), -1.0, device=sae_features.device)

thresholds_tensor = torch.tensor(thresholds, device=sae_features.device)

# Procesar cada BSP
pbar = tqdm(range(n_bsps), desc="Processing BSPs") if verbose else range(n_bsps)

for bsp_idx in pbar:
    bsp_labels = bsp_ground_truth[:, bsp_idx] # (n_positions,)

    # Skip si la BSP nunca está activa
    if not bsp_labels.any():
        continue

    # Procesar features activas en batches
    for batch_start in range(0, n_active, batch_size_features):
        batch_end = min(batch_start + batch_size_features, n_active)
        batch_features_idx = active_features[batch_start:batch_end]

        # Extraer features del batch
        batch_activations = sae_features[:, batch_features_idx] # (n_positions, batch_size)
        batch_f_max = f_max[batch_features_idx] # (batch_size,)

        # Para cada threshold, binarizar TODAS las features del batch a la vez
        for t in thresholds_tensor:
            # Binarizar: (n_positions, batch_size)
            predictions = batch_activations > (t * batch_f_max.unsqueeze(0))

            # Calcular F1 para todas las features del batch
            f1_scores = fast_f1_score_gpu(bsp_labels, predictions) # (batch_size,)

            # Encontrar la mejor en este batch
            max_f1, max_idx_in_batch = f1_scores.max(dim=0)

            # Actualizar si es mejor que lo visto hasta ahora
            if max_f1 > best_f1s[bsp_idx]:
                best_f1s[bsp_idx] = max_f1
                best_features[bsp_idx] = batch_features_idx[max_idx_in_batch]
                best_thresholds[bsp_idx] = t

        # Early stopping: si ya es casi perfecto, no revisar más batches
        if best_f1s[bsp_idx] >= 0.999:
            break

# Calcular Coverage (macro-average)
coverage = best_f1s.mean().item()

# Convertir a CPU para retornar
best_f1s_cpu = best_f1s.cpu().numpy()
best_features_cpu = best_features.cpu().numpy()
best_thresholds_cpu = best_thresholds.cpu().numpy()

return coverage, best_f1s_cpu, best_features_cpu, best_thresholds_cpu

print(" Funciones de Coverage GPU definidas")

```

Funciones de Coverage GPU definidas

0.6 6. Calcular Coverage

```
# Medir tiempo de ejecución
start_time = time.time()

coverage, best_f1s, best_features, best_thresholds = calculate_coverage_gpu_optimized(
    sae_features,
    bsp_pieces_gt_tensor,
    batch_size_features=512, # Ajustar según memoria GPU
    verbose=True
)

elapsed_time = time.time() - start_time

print("\n" + "="*60)
print("RESULTADO COVERAGE")
print("="*60)
print(f"Coverage Score: {coverage:.4f}")
print(f"Objetivo (paper): 0.52")
print(f"Diferencia: {coverage - 0.52:+.4f}")
print(f"\nTiempo de ejecución: {elapsed_time:.2f} seg")
print(f"({elapsed_time/60:.2f} min)")
print("="*60)
```

```
=====
CALCULATING COVERAGE (GPU OPTIMIZED)
=====

Positions: 11,800
SAE Features: 16,384
BSPs: 128
Thresholds: 10
Batch size (features): 512
Total evaluations: 20,971,520
=====

Active features: 9,701/16,384 (59.2%)
Reduced evaluations: 12,417,280

Processing BSPs: 100%|      | 128/128 [00:30<00:00,  4.15it/s]

=====
RESULTADO COVERAGE
=====

Coverage Score: 0.4722
Objetivo (paper): 0.52
Diferencia: -0.0478

Tiempo de ejecución: 30.91 seg
                    (0.52 min)
=====
```

0.7 7. Análisis de Resultados

```
# Estadísticas de distribución de F1 scores
print("Distribución de F1 scores:")
print(f" Mínimo: {best_f1s.min():.4f}")
print(f" Máximo: {best_f1s.max():.4f}")
print(f" Media: {best_f1s.mean():.4f}")
print(f" Mediana: {np.median(best_f1s):.4f}")
print(f" Std: {best_f1s.std():.4f}")
print()
print(f"BSPs con F1 > 0.8: {(best_f1s > 0.8).sum()} ({(best_f1s > 0.8).mean()*100:.1f}%)")
print(f"BSPs con F1 > 0.5: {(best_f1s > 0.5).sum()} ({(best_f1s > 0.5).mean()*100:.1f}%)")
print(f"BSPs con F1 < 0.2: {(best_f1s < 0.2).sum()} ({(best_f1s < 0.2).mean()*100:.1f}%)")
```

Distribución de F1 scores:

```
Mínimo: 0.3511
Máximo: 0.7582
Media: 0.4722
Mediana: 0.4495
Std: 0.0976
```

```
BSPs con F1 > 0.8: 0 (0.0%)
BSPs con F1 > 0.5: 43 (33.6%)
BSPs con F1 < 0.2: 0 (0.0%)
```

```
# Top 10 BSPs mejor detectadas
top_indices = np.argsort(best_f1s)[-10:][::-1]

print("\nTop 10 BSPs mejor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in top_indices:
    bsp_name = bsp_pieces_names[idx]
    f1 = best_f1s[idx]
    feat = best_features[idx]
    thresh = best_thresholds[idx]
    print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")
```

Top 10 BSPs mejor detectadas:

```
=====
```

BSP	F1	Feature	Threshold
BSPH82	0.7582	10178	0.3
BSPA12	0.7526	9721	0.3
BSPA82	0.7218	4712	0.3
BSPE41	0.6932	2269	0.0
BSPE51	0.6925	9114	0.0
BSPD41	0.6915	2269	0.0
BSPD51	0.6904	3343	0.0
BSPD31	0.6276	12979	0.0
BSPC51	0.6259	6034	0.0
BSPE61	0.6212	8606	0.0

```
-----
```

```
# Bottom 10 BSPs peor detectadas
bottom_indices = np.argsort(best_f1s)[:10]

print("\nBottom 10 BSPs peor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in bottom_indices:
```



```

bsp_name = bsp_pieces_names[idx]
f1 = best_fis[idx]
feat = best_features[idx]
thresh = best_thresholds[idx]
print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")

```

Bottom 10 BSPs peor detectadas:

```

=====
BSP          F1          Feature    Threshold
-----
BSPC82       0.3511       6955         0.2
BSPH32       0.3581       6955         0.2
BSPA52       0.3594       6955         0.1
BSPD82       0.3595       8227         0.1
BSPE12       0.3614       8928         0.2
BSPH42       0.3618       1515         0.2
BSPB11       0.3649       6955         0.2
BSPH52       0.3651       6955         0.2
BSPE82       0.3652       642          0.1
BSPF11       0.3656       6955         0.2

```

```

# Distribución de thresholds óptimos
unique_thresholds, counts = np.unique(best_thresholds, return_counts=True)

print("\nDistribución de thresholds óptimos:")
print("="*60)
for t, count in zip(unique_thresholds, counts):
    if t >= 0: # Ignorar -1 (BSPs sin match)
        percentage = count / len(best_thresholds) * 100
        print(f"Threshold {t:.1f}: {count:3d} BSPs ({percentage:.1f}%)")

```

Distribución de thresholds óptimos:

```

=====
Threshold 0.0:  57 BSPs (44.5%)
Threshold 0.1:  20 BSPs (15.6%)
Threshold 0.2:  42 BSPs (32.8%)
Threshold 0.3:   9 BSPs (7.0%)

```

```

# =====
# CONSULTA POR BSP ESPECÍFICA
# =====

# >>> CAMBIA AQUÍ LA BSP QUE QUIERES ANALIZAR <
BSP_OBJETIVO = "BSPH82"

# Buscar índice de la BSP
bsp_idx = bsp_pieces_names.index(BSP_OBJETIVO)

# Extraer resultados ya calculados
neurona  = best_features[bsp_idx]
threshold = best_thresholds[bsp_idx]
f1       = best_fis[bsp_idx]

# f_max de la neurona ganadora (sae_features ya está en memoria)
f_max_neurona = sae_features[:, neurona].max().item()

# Activaciones de esa neurona en todos los tableros
activaciones_neurona = sae_features[:, neurona].cpu().numpy()

```

```

print(f"BSP analizada:      {BSP_OBJETIVO}")
print(f"Neurona ganadora:  {neurona}")
print(f"Threshold óptimo:   {threshold:.1f}")
print(f"F_max neurona:      {f_max_neurona:.4f}")
print(f"Umbral efectivo:    {threshold:.1f} x {f_max_neurona:.4f} = {threshold * f_max_neurona:.4f}")
print(f"F1 score:          {f1:.4f}")
print(f"\nActivaciones de neurona {neurona} ({len(activaciones_neurona)} tableros):")
print(activaciones_neurona)

```

```

BSP analizada:      BSPH82
Neurona ganadora:  10178
Threshold óptimo:   0.3
F_max neurona:      1.4170
Umbral efectivo:    0.3 x 1.4170 = 0.4251
F1 score:          0.7582

```

```

Activaciones de neurona 10178 (11800 tableros):
[0.      0.      0.      ... 0.4347538  0.12395656 0.6656618 ]

```

0.8 8. Guardar Resultados

```

# Guardar resultados en el directorio de métricas del experimento
results = {
    'coverage': coverage,
    'best_fls': best_fls,
    'best_features': best_features,
    'best_thresholds': best_thresholds,
    'bsp_names': bsp_pieces_names,
    'execution_time_seconds': elapsed_time
}

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos']
)
output_path = metrics_dir / "coverage_results.npz"
np.savez(output_path, **results)

print(f" Resultados guardados en:")
print(f"   {output_path}")

```

Resultados guardados en:

```
c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1_standard\1000games\metrics
```

0.9 9. Generar PDF con Quarto

```

import subprocess

# Guardar PDF en el mismo directorio que el .npz (metrics/)
pdf_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos']
)

```

```

)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'coverage'
)

notebook_name = 'coverage_gpu_optimized.ipynb'
output_path = pdf_dir / pdf_name

print(f' Generando PDF con Quarto...')
print(f' Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{pdf_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f' PDF generado exitosamente: {output_path}')
else:
    print(f' Error al generar PDF:')
    print(result.stderr)

```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1_standard

PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1_standard